homework7

2026-10-10

Homework 7

Question 1

a) The empirical priors are $\hat{\pi}_0 = 0.6537$ and $\hat{\pi}_1 = 0.3463$. The class means are:

Class	sbp	tobacco	ldl	adiposity	famhist	typea	obesity	alcohol	age
0	135.460	2.635	4.344	23.969	1.318	52.368	25.737	15.931	38.854
1	143.738	5.525	5.488	28.120	1.600	54.494	26.623	19.145	50.294

Using denominator $n - K = 460$, the pooled covariance matrix is:

	sbp	tobacco	ldl	adiposity	famhist	typea	obesity	alcohol	age
sbp	405.435	14.585	4.581	49.164	0.337	-15.588	18.939	64.400	95.134
tobacco	14.585	19.243	0.763	7.538	0.016	-2.057	1.833	20.517	22.765
ldl	4.581	0.763	4.001	6.033	0.092	0.345	2.660	-2.533	6.479
adiposity	49.164	7.538	6.033	56.753	0.433	-5.310	22.708	16.119	60.509
famhist	0.337	0.016	0.092	0.433	0.226	0.081	0.184	0.768	0.997
typea	-15.588	-2.057	0.345	-5.310	0.081	95.566	2.640	7.960	-20.278
obesity	18.939	1.833	2.660	22.708	0.184	2.640	17.615	4.689	15.697
alcohol	64.400	20.517	-2.533	16.119	0.768	7.960	4.689	598.277	27.886
age	95.134	22.765	6.479	60.509	0.997	-20.278	15.697	27.886	184.132

b) The dimensions are $X : 462 \times 9$, means: 2×9, pooled covariance: 9×9, score matrix: 462×2, and fitted labels: length 462. The manual LDA scores give:

Observed \ Predicted	0	1
0	258	44
1	73	87

c) `LinearDiscriminantAnalysis(solver="svd")` gives the same confusion matrix, so the manual calculation and the library implementation match exactly.

```
import numpy as np, pandas as pd
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

heart = pd.read_csv("data/saheart.csv")
heart["famhist"] = heart["famhist"].map({"Absent": 1, "Present": 2})
X = heart.drop(columns="chd").to_numpy(float)
y = heart["chd"].to_numpy(int)
classes = np.array([0, 1]); n, p = X.shape
priors = np.array([(y == k).mean() for k in classes])
means = np.array([X[y == k].mean(axis=0) for k in classes])

S = np.zeros((p, p))
for j, k in enumerate(classes):
```

```
    Z = X[y == k] - means[j]
    S += Z.T @ Z
S /= (n - len(classes))

scores = []
for j in range(2):
    Sinv_mu = np.linalg.solve(S, means[j])
    scores.append(X @ Sinv_mu - .5 * means[j] @ Sinv_mu + np.log(priors[j]))
pred_manual = classes[np.argmax(np.column_stack(scores), axis=1)]

lda = LinearDiscriminantAnalysis(solver="svd", priors=priors).fit(X, y)
pred_library = lda.predict(X)
```

Question 2

a) With the 40 training pixels having the largest marginal variances, QDA fits successfully. The test misclassification rate is **0.1555**.

Observed \ Predicted	0	1	2	3	4	5	6	7	8	9
0	319	0	24	6	2	2	1	0	5	0
1	0	238	5	0	11	0	7	0	3	0
2	8	0	168	8	8	3	0	0	3	0
3	5	0	4	128	0	14	0	0	15	0
4	0	2	13	1	150	1	0	1	7	25
5	12	0	3	5	1	131	0	0	6	2
6	0	0	8	0	1	8	148	0	5	0
7	1	0	6	1	6	0	0	124	2	7
8	1	0	10	12	3	2	0	0	137	1
9	0	0	1	2	7	1	1	4	9	152

b) Yes. Reducing from 256 to 40 predictors makes the within-class covariance estimates manageable and removes many nearly constant pixels, so the class covariance matrices can be estimated without the singularity problem seen with all pixels.

c) Using only the 10 highest-variance pixels increases the test misclassification rate to **0.3423**.

Observed \ Predicted	0	1	2	3	4	5	6	7	8	9
0	298	0	11	1	1	16	28	4	0	0
1	0	241	10	0	3	1	1	0	2	6
2	19	2	95	14	14	3	22	21	5	3
3	5	0	6	103	3	42	0	0	7	0
4	1	5	14	0	95	2	4	15	17	47
5	15	0	6	25	4	96	4	2	8	0
6	41	0	15	4	0	18	78	7	5	2
7	4	1	6	0	6	0	1	122	0	7
8	1	2	11	36	12	11	4	1	74	14
9	0	2	9	2	20	0	0	16	10	118

The smaller covariance matrices are easier to estimate, but ten marginally variable pixels discard too much useful digit information. Marginal variance is not class-aware, so aggressive screening can remove pixels that help separate digits.

```
import numpy as np, pandas as pd
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis

train = pd.read_csv("data/zip-train.csv.gz", header=None)
test  = pd.read_csv("data/zip-test.csv.gz", header=None)
Xtr, ytr = train.iloc[:, 1:].to_numpy(), train.iloc[:, 0].to_numpy(int)
Xte, yte = test.iloc[:, 1:].to_numpy(), test.iloc[:, 0].to_numpy(int)

marginal_var = Xtr.var(axis=0, ddof=1)

def run_qda(m):
    keep = np.argsort(marginal_var)[-m:]
    fit = QuadraticDiscriminantAnalysis(tol=1e-8).fit(Xtr[:, keep], ytr)
    pred = fit.predict(Xte[:, keep])
    cm = np.array([[np.sum((yte == i) & (pred == j)) for j in range(10)]
                   for i in range(10)])
    return cm, np.mean(pred != yte)

cm40, error40 = run_qda(40)
cm10, error10 = run_qda(10)
```

Question 3

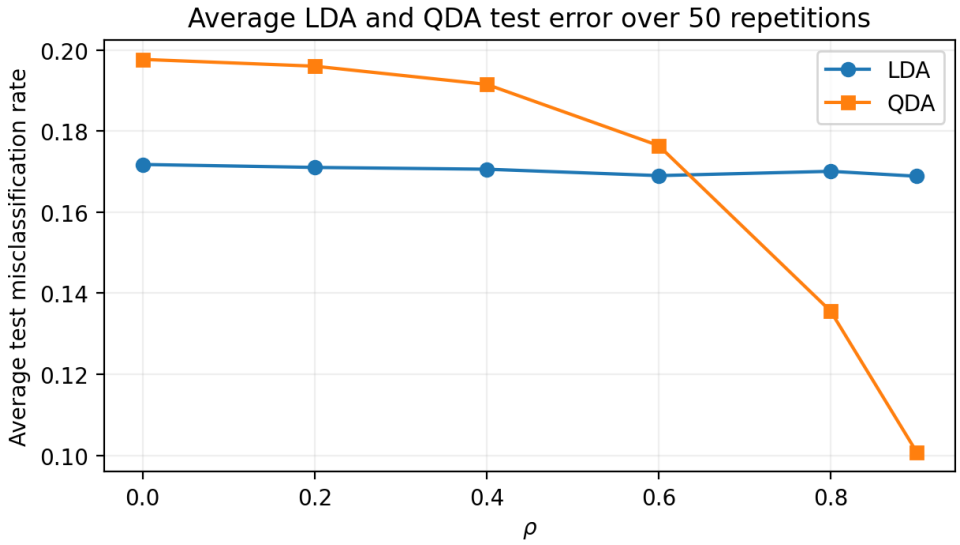
Seed used for part (a): **43207**. The same generated training and test observations were used for LDA and QDA within each value of ρ.

a)

ρ	LDA test error	QDA test error
0.0	0.1815	0.2085
0.2	0.1835	0.1975
0.4	0.1740	0.1905
0.6	0.1585	0.1575
0.8	0.1670	0.1395
0.9	0.1775	0.1005

b) For the 50-repetition experiment, seed **432070** was used.

ρ	Mean LDA error	Mean QDA error
0.0	0.17176	0.19768
0.2	0.17103	0.19600
0.4	0.17059	0.19152
0.6	0.16901	0.17642
0.8	0.17007	0.13556
0.9	0.16886	0.10080



c) At small ρ, the class covariances are equal or nearly equal, so LDA has lower error because its pooled covariance estimate has less variance. As ρ becomes large, the class covariance structures differ strongly and QDA becomes better because its separate covariance matrices reduce model bias. The eight noise variables still increase estimation noise, especially for QDA because it estimates more covariance parameters.

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis,
    QuadraticDiscriminantAnalysis
import numpy as np

rhos = [0, .2, .4, .6, .8, .9]

def generate(rng, rho, ntrain=100, ntest=1000):
    p = 10
```

```
    S1 = np.eye(p); S2 = np.eye(p)
    S1[0,1] = S1[1,0] = rho
    S2[0,1] = S2[1,0] = -rho
    m1 = np.zeros(p); m1[0] = -1
    m2 = np.zeros(p); m2[0] = 1
    A = rng.multivariate_normal(m1, S1, ntrain)
    B = rng.multivariate_normal(m2, S2, ntrain)
    C = rng.multivariate_normal(m1, S1, ntest)
    D = rng.multivariate_normal(m2, S2, ntest)
    return (np.vstack([A,B]), np.r_[np.ones(ntrain), 2*np.ones(ntrain)],
            np.vstack([C,D]), np.r_[np.ones(ntest), 2*np.ones(ntest)])

def errors(Xtr, ytr, Xte, yte):
    lda = LinearDiscriminantAnalysis(priors=[.5,.5]).fit(Xtr, ytr)
    qda = QuadraticDiscriminantAnalysis(priors=[.5,.5], tol=1e-8).fit(Xtr, ytr)
    return np.mean(lda.predict(Xte) != yte), np.mean(qda.predict(Xte) != yte)

rng = np.random.default_rng(432070)
average = []
for rho in rhos:
    e = [errors(*generate(rng, rho)) for _ in range(50)]
    average.append((rho, *np.mean(e, axis=0)))
```

Question 4

a) I created the reusable `lda-qda-analysis-report` skill. The final `SKILL.md` is **2,392 bytes**, below the 5,000-byte limit. It is configured for explicit invocation only and requires ordinary LDA and QDA, shared CV folds, fold-specific preprocessing, an untouched test set, and a one-page report.

The skill was kept outside automatically loaded skill folders and supplied only when explicitly requested.

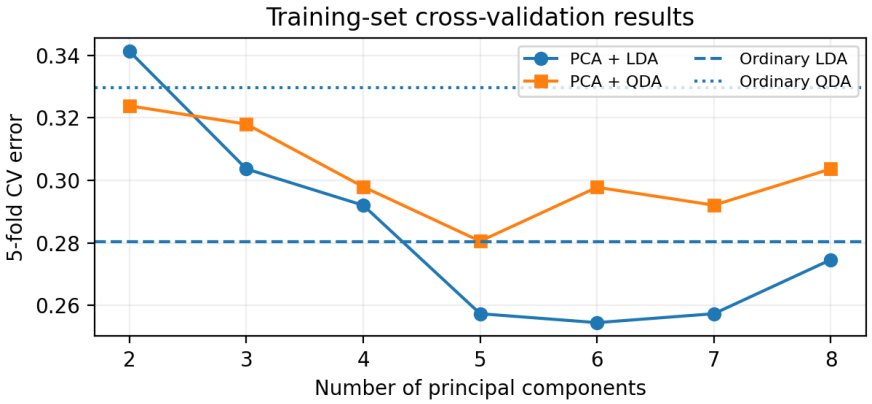
Prompt used to apply the skill:

> Use the attached `lda-qda-analysis-report` skill to analyze the SAheart data. Use `chd` as the response and the same nine predictors and famhist coding as Question 1. Seek the best predictive performance using only LDA/QDA-based candidates. Reserve a stratified test set before model selection, use shared training-set CV folds, estimate all preprocessing inside each fold, compare by misclassification rate, report the random seed, and return the required one-page report.

Question 4(b) - One-page analysis report

Data and validation. Seed **4327**. A stratified 75/25 split produced 346 training observations (226 class 0, 120 class 1) and 116 untouched test observations (76 class 0, 40 class 1). Five-fold stratified CV on the training set used identical folds for every candidate; scaling and PCA were refit inside each fold.

Candidate (best setting within family)	Mean CV error
Ordinary LDA	0.2804
Ordinary QDA	0.3297
PCA + LDA (6 PCs)	**0.2545**
PCA + QDA (5 PCs)	0.2806
Probability blend (25% QDA, 75% LDA)	0.2920



Selected model and final test result. PCA with 6 components followed by LDA had the lowest mean CV error. Refit on all training observations, the six PCs explained 85.3% of standardized predictor variance. On the untouched test set, the misclassification rate was **0.2759** (32/116 errors).

Observed \ Predicted	0	1
0	58	18
1	14	26

The CV results favor dimension reduction before LDA, while ordinary QDA has the largest CV error, consistent with higher covariance-estimation variance in this modest sample. The final error is based on one holdout split, so it remains sensitive to the random split and class imbalance; the test set was not used to revise the model or skill.

Question 4(c)

No revisions were made using the test result. The final skill contents are:

```
---
name: lda-qda-analysis-report
description: Run only when explicitly invoked by name or when this SKILL.md is supplied in
    the request. Analyze a classification dataset using LDA/QDA-based models and return a
    concise one-page statistical report.
---

# LDA/QDA Analysis Report Skill

Use this skill only on explicit invocation. The user will provide the dataset, response,
    predictors, and any coding rules separately.

## Analysis workflow
1. Validate the response classes, predictor dimensions, missing values, and class counts. Do
    not silently change the requested variables or coding.
2. Before model selection, reserve a stratified test set and report the random seed. Do not
    inspect test-set performance while choosing models.
3. On the remaining training data, create one shared set of stratified cross-validation folds
     for every candidate.
4. Always include ordinary LDA and ordinary QDA. You may also compare:
   - PCA followed by LDA;
   - PCA followed by QDA;
   - a convex blend of LDA and QDA class probabilities.
   Keep every classifier based on LDA, QDA, or their combination.
5. Estimate every preprocessing step inside each training fold only. If scaling or PCA is
    used, fit it on the fold's training rows and apply that fitted transformation to the
    validation rows.
6. Use misclassification rate as the model-selection metric unless the user explicitly
    requests another metric. Choose the candidate/settings with the lowest mean CV error;
    break exact ties in favor of the simpler model.
7. Refit the selected preprocessing and classifier on all training observations. Evaluate it
    once on the untouched test set.
8. Check for numerical problems such as singular covariance matrices, tiny classes, or
    unstable QDA estimates. If a candidate fails, report the failure instead of hiding it.

## Required one-page report
Produce at most one printed page containing:
- a short data/split statement with seed and class counts;
- a compact table of candidate models and cross-validation error;
- one readable figure showing the main model-comparison result;
- the selected model and its settings;
- the selected model's test misclassification rate and a small confusion matrix;
- 2-4 sentences explaining the main finding and limitations.

Keep the report concise. Distinguish training/CV results from final test results. Do not
    choose or revise a model based on test performance.
```